NHA TRANG UNIVERSITY
Faculty of Information Technology

CHAPTER 1
Introduction to Java Programming

Course: Java Programming (NEC331) — 3 Credits (30 Lecture + 30 Lab)

👨‍🏫 Instructor: **Pham Van Nam** ☕ JDK 23 🗓️ 2025

📋 Table of Contents

🎯 LEARNING OUTCOMES

Upon completing Chapter 1, students will be able to:

- **LO1.1** Describe the history and key features of Java; distinguish the components JDK, JRE, and JVM and their roles in the compilation–execution process. *(Remember, Understand)*
- **LO1.2** Install JDK 23, configure the development environment, and compile and run a Java program. *(Apply)*
- **LO1.3** Correctly use primitive data types, variables, constants, operators, type casting, and distinguish between Stack and Heap memory. *(Understand, Apply)*
- **LO1.4** Proficiently apply control flow statements (if/else, switch, for, while, do-while) to solve logic-based problems. *(Apply)*
- **LO1.5** Declare, initialize, and manipulate arrays, String, and StringBuilder; differentiate String vs StringBuilder. *(Apply, Analyze)*

> ⚠ **Prerequisites:** Students should have a solid understanding of fundamental programming concepts (variables, loops, functions) from an introductory programming course (C/C++ or Python).

1.1. INTRODUCTION TO THE JAVA LANGUAGE

1.1.1. History and Origins

Java is a high-level , object-oriented programming language developed by **James Gosling** and his team at **Sun Microsystems** in the early 1990s. Originally named *"Oak"*, the language was designed for embedded devices . In **1995**, it was renamed "Java" and officially released to the public.

In **2010**, Oracle Corporation acquired Sun Microsystems and has maintained Java ever since. Starting from JDK 10 (2018), Oracle adopted a **6-month release cycle**. **Long-Term Support (LTS)** versions such as JDK 11, 17, and 21 receive extended support and are preferred in enterprise environments.

Notable Milestones

Version	Year	Key Highlights
`JDK 1.0`	1996	First release
`JDK 5`	2004	Generics, Enum, Autoboxing, Enhanced for
`JDK 8` ⭐	2014	Lambda Expressions, Stream API — shift toward functional programming
`JDK 9`	2017	Module System, JShell
`JDK 14`	2020	Switch Expressions finalized
`JDK 15`	2020	Text Blocks
`JDK 16`	2021	Records, Pattern Matching for `instanceof`
`JDK 21` ⭐	2023	LTS — Virtual Threads, Sequenced Collections
`JDK 23`	2024	Version used in this course

1.1.2. Java Platform Editions

Edition	Description	Target Audience
Java SE (Standard Edition)	Core platform providing fundamental APIs — *used in this course*	All Java developers
Jakarta EE (formerly Java EE)	Extends Java SE for enterprise applications: Servlet, JSP, JPA, EJB	Web/enterprise development
Java ME (Micro Edition)	Trimmed-down edition for embedded devices and IoT (now rarely used)	Resource-constrained devices
Android SDK	Uses Java/Kotlin syntax but runs on ART instead of JVM	Android app development

1.1.3. Key Features of Java

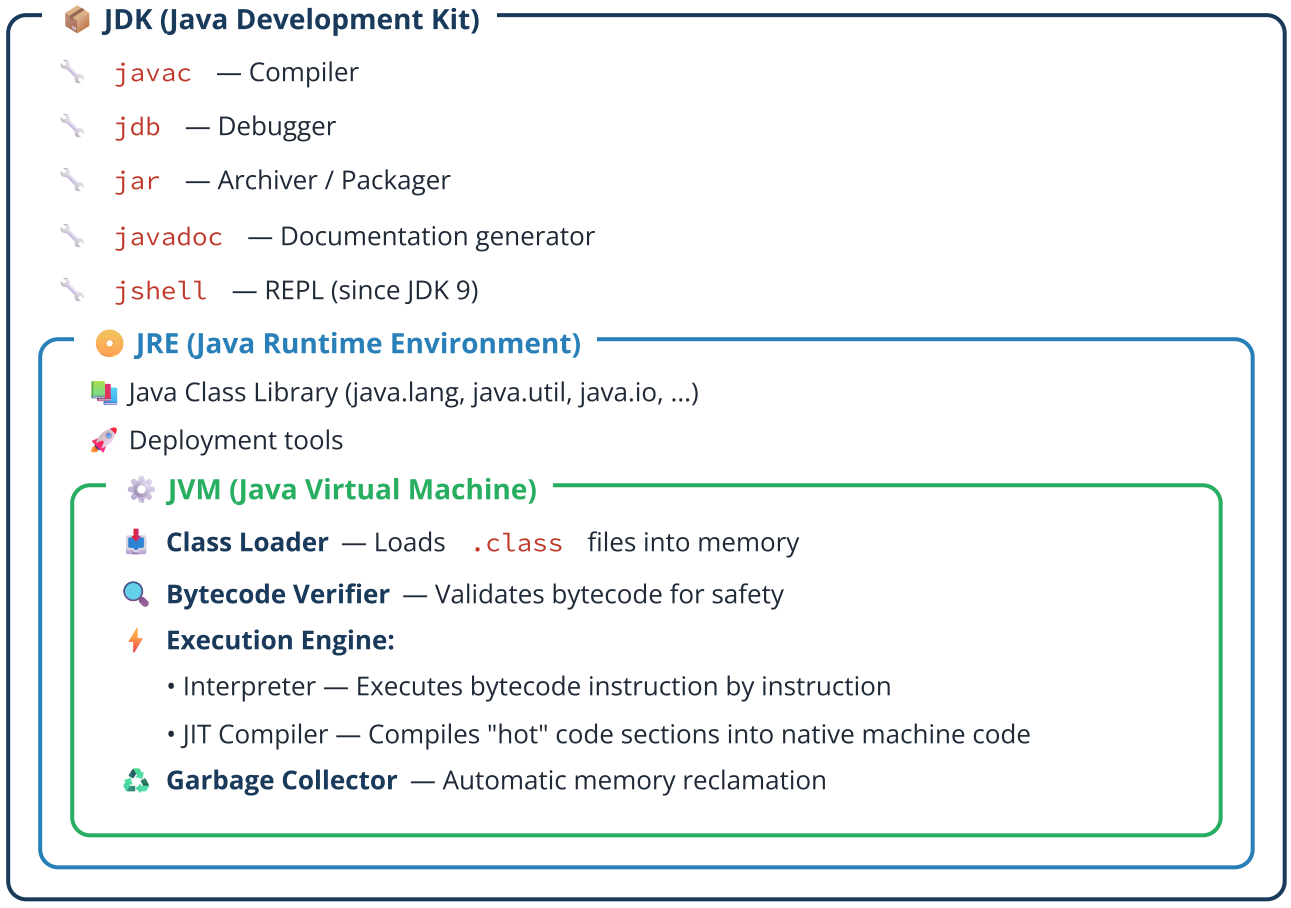
Java embodies the famous design philosophy: **"Write Once, Run Anywhere" (WORA)**. Java source code is compiled into bytecode that can run on any platform with a JVM.

Feature	Description
Object-Oriented	Everything revolves around classes and objects. Fully supports: encapsulation , inheritance , polymorphism , and abstraction .
Automatic Memory Management	The Garbage Collector automatically reclaims objects with no remaining references — eliminating memory leaks.
Statically Typed	Every variable must be declared with a type at compile time. Errors are caught early.

Feature	Description
Multithreaded	Built-in support for `Thread` and `Runnable`. JDK 21+ introduces Virtual Threads.
Secure	JVM sandbox, bytecode verifier, layered ClassLoader architecture.
Rich Ecosystem	Spring, Hibernate, Quarkus, and thousands of open-source libraries.

1.1.4. Platform Architecture: JDK, JRE, JVM

Three core concepts with a nested relationship: **JDK** ⊃ **JRE** ⊃ **JVM**.



> 💡 **Note**
>
> Since JDK 11, Oracle no longer distributes a standalone JRE — installing the JDK includes everything. The `jlink` tool allows creating custom runtime images containing only the required modules.

1.1.5. Installing JDK 23 and Configuring the Environment

Step 1: Download JDK 23 from oracle.com/java/technologies/downloads or OpenJDK at jdk.java.net/23.

Step 2: Install the JDK following your operating system's instructions.

Step 3: Set up environment variables.

Operating System	Configuration
Windows	Add `JAVA_HOME = C:\Program Files\Java\jdk-23` Append `%JAVA_HOME%\bin` to `PATH`

Linux / macOS	Add to `~/.bashrc` or `~/.zshrc`:
	`export JAVA_HOME=/path/to/jdk-23`
	`export PATH=$JAVA_HOME/bin:$PATH`

Step 4: Verify the installation:

Terminal

```
java --version
javac --version
```

Recommended IDE: IntelliJ IDEA Community Edition (free, the most powerful IDE for Java). Students can register a JetBrains Education account to use the Ultimate edition for free.

1.1.6. Compilation and Execution Process

Java combines both models: **compilation** (source code → bytecode) and **interpretation** (bytecode → machine code). This achieves both platform independence and good performance thanks to the JIT Compiler.

1.1.7. Your First Java Program

HelloWorld.java

```
// File: HelloWorld.java
// Each .java file contains at most ONE public class
// The filename MUST match the public class name (case-sensitive)

public class HelloWorld {
    // The main method: the entry point for program execution
    public static void main(String[] args) {
        System.out.println("Hello! Welcome to Java.");
        System.out.println("JDK version: " + System.getProperty("java.version"));
        System.out.println("Operating system: " + System.getProperty("os.name"));
    }
}
```

Detailed Analysis of Each Component

Component	Explanation
`public class HelloWorld`	Declares a public class. The filename **must** match the class name (case-sensitive).
`public`	Allows JVM to access the method from outside the class.
`static`	Can be called without creating an object instance.
`void`	The method does not return a value.
`String[] args`	Array of command-line arguments.
`System.out.println(...)`	`System` : class in `java.lang` ; `out` : standard output stream; `println` : prints + newline.

Compiling and Running from the Command Line

Terminal

```
# Compile: creates the file HelloWorld.class
javac HelloWorld.java

# Run: JVM executes the bytecode (do NOT include .class)
java HelloWorld

# Since JDK 11: run a .java file directly (convenient for learning)
java HelloWorld.java
```

Running with Command-Line Arguments

Greeting.java

```
public class Greeting {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Usage: java Greeting <your_name>");
            return;
        }

        System.out.println("Hello, " + args[0] + "!");
        System.out.println("You passed " + args.length + " argument(s):");
        for (int i = 0; i < args.length; i++) {
            System.out.println("  args[" + i + "] = \"" + args[i] + "\"");
        }
    }
}
```

1.1.8. Exploring Java with JShell (REPL)

Since JDK 9, Java provides **JShell** — a **Read-Eval-Print Loop** that lets you execute individual Java statements interactively. This is extremely convenient for learning and experimentation:

JShell

```
$ jshell
|  Welcome to JShell -- Version 23

jshell> int x = 10;
x ==> 10

jshell> int y = 20;
y ==> 20

jshell> System.out.println("Sum = " + (x + y));
Sum = 30

jshell> Math.sqrt(144)
$4 ==> 12.0

jshell> "Nha Trang".toUpperCase()
$5 ==> "NHA TRANG"

jshell> /exit
```

1.1.9. Packages and the `import` Statement

PackageDemo.java

```java
// Package declaration: must be the first line in the file
package com.ntu.javacourse.chapter1;

// Import a specific class
import java.util.Scanner;
import java.util.Arrays;

// Import all classes in a package (discouraged — less explicit)
// import java.util.*;

public class PackageDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = {3, 1, 4, 1, 5};
        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));
        sc.close();
    }
}
```

> 💡 **Package Naming Convention**
>
> Use the reversed domain name convention: `com.ntu.javacourse.chapter1` , `org.apache.commons.lang3` . Package names are written entirely in lowercase. The `java.lang` package is imported automatically.

1.1.10. Naming Conventions in Java

Element	Convention	Examples
Class / Interface	`PascalCase`	`Student` , `HelloWorld` , `ArrayList`
Variable / Method	`camelCase`	`fullName` , `averageScore` , `getName()`
Constant	`UPPER_SNAKE_CASE`	`MAX_SIZE` , `PI` , `DEFAULT_TIMEOUT`
Package	`all lowercase`	`java.util` , `com.ntu.app`

Names should be meaningful and clearly describe their purpose. Use `studentCount` instead of `sc` or `x` .

1.1.11. Comments

Comments.java

```java
// Single-line comment

/* Multi-line comment
   Used for longer explanations */

/**
 * Javadoc comment
 * Used to generate API documentation automatically via javadoc.
 *
 * @author Pham Van Nam
 * @version 1.0
 * @since 2025
 */
public class DocExample {
    /**
     * Calculates the sum of two integers.
     *
     * @param a the first integer
     * @param b the second integer
     * @return the sum of a and b
     */
    public static int calculateSum(int a, int b) {
        return a + b;
    }
}
```

✅ Principles of Good Commenting

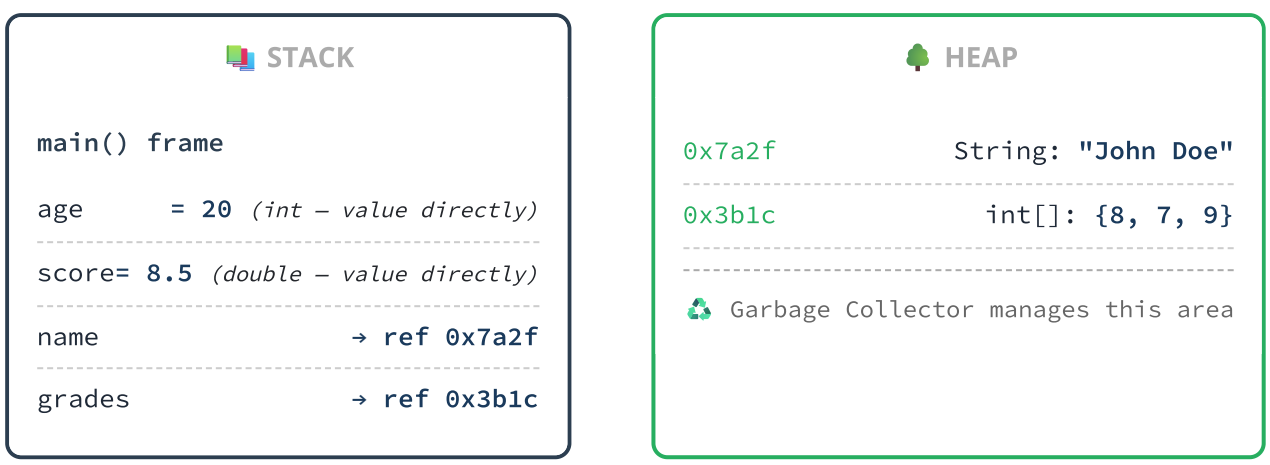
Comments should explain *why*, not repeat *what* the code already shows. For example, `i++; // increment i by 1` is useless. However, `i++; // move to the next node in the adjacency list` is meaningful.

1.2. PRIMITIVE DATA TYPES

1.2.1. Basic Memory Model: Stack and Heap

The **Stack** stores local variables and method call frames. It is small in size, fast to access, and automatically deallocated when a method returns.

The **Heap** stores objects created with `new` and String literals. It is larger and managed by the **Garbage Collector** .



> ⚠️ **Critical Rule**
>
> **Primitive types** (`int` , `double` , `boolean` ...) → store the **value directly** on the Stack.
> **Reference types** (`String` , arrays, objects) → store a **reference (address)** on the Stack; the actual object resides on the Heap.

1.2.2. Variables

A **variable** is a named memory location used to store data. Java is a **statically typed** language — every variable must have its type declared at compile time.

Variable Type	Declared In	Default Value	Scope
Local Variable	Inside a method or block `{}`	*None* — must be initialized	Within the method/block
Instance Variable	In a class, outside any method	0 / 0.0 / false / null	The entire object
Static Variable	With the `static` keyword	0 / 0.0 / false / null	The entire class (shared)

VariablesAndData.java

```java
public class VariablesAndData {
    public static void main(String[] args) {
        // --- Declaring and initializing local variables ---
        int age = 20;
        double gpa = 8.5;
        char grade = 'A';
        boolean hasPassed = true;
        String fullName = "John Doe";   // String is a reference type

        System.out.println("Name: " + fullName);
        System.out.println("Age: " + age);
        System.out.println("GPA: " + gpa);

        // Declare first, assign later (valid if assigned BEFORE use)
        int absences;
        absences = 3;

        // Declare multiple variables of the same type
        int a = 1, b = 2, c = 3;
        System.out.println("a + b + c = " + (a + b + c));

        // Variable scope
        {
            int blockVariable = 100;
            System.out.println("Inside block: " + blockVariable);
        }
        // System.out.println(blockVariable); // ERROR: out of scope!
    }
}
```

The `var` Keyword — Local Variable Type Inference JDK 10+

VarDemo.java

```java
var name = "Alice Johnson";         // Inferred: String
var score = 9.5;                    // Inferred: double
var count = 100;                    // Inferred: int
var numbers = new int[]{1, 2, 3};   // Inferred: int[]

// ✅ Use when the type is clear from the right-hand side:
var scanner = new Scanner(System.in);

// ❌ Avoid when the type is unclear:
// var result = calculate();  // Reader doesn't know the return type
```

1.2.3. Constants

Constants.java

```java
public class Constants {
    // Class-level constants (static final — UPPER_SNAKE_CASE)
    static final double PI = 3.14159265358979;
    static final int MONTHS_PER_YEAR = 12;
    static final String UNIVERSITY_NAME = "Nha Trang University";

    public static void main(String[] args) {
        final int MAX_STUDENTS = 50;   // Local constant

        System.out.println("University: " + UNIVERSITY_NAME);
        System.out.println("Circumference (R=5): " + (2 * PI * 5));

        // MAX_STUDENTS = 60;  // ERROR: cannot assign a value to final variable
    }
}
```

1.2.4. The Eight Primitive Data Types

Category	Type	Size	Range	Default
Integer	`byte`	8-bit	-128 to 127	0
	`short`	16-bit	-32,768 to 32,767	0
	`int`	32-bit	≈ ±2.1 billion	0
	`long`	64-bit	$\pm 9.2 \times 10^{18}$	0L
Floating-point	`float`	32-bit	6–7 decimal digits precision	0.0f
	`double`	64-bit	15–16 decimal digits precision	0.0
Character	`char`	16-bit	'\u0000' to '\uffff' (Unicode)	'\u0000'
Boolean	`boolean`	~1-bit	`true` / `false`	false

PrimitiveTypes.java

```
public class PrimitiveTypes {
    public static void main(String[] args) {
        // ===== INTEGER TYPES =====
        byte smallNum = 127;
        short mediumNum = 32_000;              // Underscores for readability (Java 7+)
        int counter = 1_000_000;               // Default type for integer literals
        long population = 8_000_000_000L;      // Suffix L is required for long

        // Number bases
        int hex = 0xFF;          // Hexadecimal: 255
        int binary = 0b1010;     // Binary: 10
        int octal = 0777;        // Octal: 511

        // ===== FLOATING-POINT TYPES =====
        float examScore = 8.5f;                // Suffix f is required for float
        double pi = 3.141592653589793;         // Default type for decimal literals
        double scientific = 1.5e10;            // Scientific notation: 1.5 × 10^10

        // ===== CHARACTER TYPE =====
        char letter = 'A';
        char unicodeChar = '\u0041';           // Also 'A'
        char vietnameseChar = '\u1EC6';        // Java fully supports Unicode
        System.out.println("'A' + 1 = " + (char)('A' + 1));  // 'B'

        // ===== BOOLEAN TYPE =====
        boolean isLearningJava = true;
        boolean result = (10 > 5);             // true

        // ===== RANGE INFORMATION =====
        System.out.println("int  : " + Integer.MIN_VALUE + " to " + Integer.MAX_VALUE);
        System.out.println("long : " + Long.MIN_VALUE + " to " + Long.MAX_VALUE);
    }
}
```

> 🤔 **Thinking Question**
>
> When should you use `int` vs `long`? When `float` vs `double`?
>
> **Rule of thumb:** Use `int` + `double` as defaults. Use `long` when values exceed ±2.1 billion. Use `float` when memory savings matter with large arrays.

1.2.5. Common Pitfalls: Integer Overflow & Floating-Point Precision

CommonPitfalls.java

```java
public class CommonPitfalls {
    public static void main(String[] args) {
        // ===== INTEGER OVERFLOW =====
        int maxInt = Integer.MAX_VALUE;  // 2,147,483,647
        System.out.println("Max int + 1 = " + (maxInt + 1));  // -2,147,483,648 ← overflow!

        // Java does NOT throw an error on overflow — it wraps around silently!
        // Solution: use Math.addExact() to detect overflow
        try {
            int result = Math.addExact(maxInt, 1);
        } catch (ArithmeticException e) {
            System.out.println("Overflow detected: " + e.getMessage());
        }

        // ===== FLOATING-POINT PRECISION LOSS =====
        System.out.println("0.1 + 0.2 = " + (0.1 + 0.2));     // 0.30000000000000004
        System.out.println("0.1 + 0.2 == 0.3? " + (0.1 + 0.2 == 0.3));  // false!

        // Solution: compare with epsilon
        double epsilon = 1e-9;
        boolean closeEnough = Math.abs(0.1 + 0.2 - 0.3) < epsilon;
        System.out.println("Epsilon comparison: " + closeEnough);  // true

        // ===== INTEGER DIVISION =====
        System.out.println("7 / 2 = " + (7 / 2));           // 3 (truncates decimal!)
        System.out.println("7.0 / 2 = " + (7.0 / 2));       // 3.5
        System.out.println("(double)7 / 2 = " + ((double) 7 / 2)); // 3.5
    }
}
```

1.2.6. Type Casting

Type	Direction	Syntax	Risk
Widening	Smaller → Larger `byte→short→int→long→float→double`	Automatic (implicit)	Safe (except long→float)
Narrowing	Larger → Smaller	Explicit: `(type) value`	Possible data loss!

TypeCasting.java

```java
public class TypeCasting {
    public static void main(String[] args) {
        // ===== WIDENING (automatic) =====
        int intValue = 100;
        long longValue = intValue;       // int → long
        double doubleValue = longValue;  // long → double

        // ===== NARROWING (must be explicit) =====
        double originalScore = 8.75;
        int truncated = (int) originalScore;       // 8 (TRUNCATES, does not round!)
        long rounded = Math.round(originalScore);  // 9 (rounds correctly)

        // Beware of overflow during narrowing
        int largeInt = 130;
        byte narrowed = (byte) largeInt;           // -126 (overflow!)

        // ===== TYPE PROMOTION IN EXPRESSIONS =====
        byte b1 = 10, b2 = 20;
        // byte b3 = b1 + b2;    // ERROR! Result is promoted to int
        int b3 = b1 + b2;        // Correct
        byte b4 = (byte)(b1 + b2); // Or cast explicitly

        // Rules: if double present → double; if float → float;
        //        if long → long; otherwise → int
        double r1 = 5 / 2;         // 2.0 (integer division FIRST, then promotion)
        double r2 = 5.0 / 2;       // 2.5 (2 promoted to double first)
        double r3 = (double) 5 / 2; // 2.5

        // ===== char ↔ int =====
        char ch = 'A';
        int code = ch;               // 65 (widening)
        char fromCode = (char) 66;   // 'B' (narrowing)
    }
}
```

1.2.7. Operators

Operators.java

```
public class Operators {
    public static void main(String[] args) {
        // ===== ARITHMETIC =====
        int a = 17, b = 5;
        System.out.println("a + b  = " + (a + b));    // 22
        System.out.println("a - b  = " + (a - b));    // 12
        System.out.println("a * b  = " + (a * b));    // 85
        System.out.println("a / b  = " + (a / b));    // 3 (integer division!)
        System.out.println("a % b  = " + (a % b));    // 2 (remainder)

        // ===== INCREMENT / DECREMENT =====
        int x = 10;
        System.out.println("x++  = " + (x++));  // Prints 10, then x becomes 11 (postfix)
        System.out.println("++x  = " + (++x));  // x becomes 12, then prints 12 (prefix)

        // ===== RELATIONAL (COMPARISON) =====
        System.out.println("10 == 20: " + (10 == 20));  // false
        System.out.println("10 != 20: " + (10 != 20));  // true
        System.out.println("10 <= 20: " + (10 <= 20));  // true

        // ===== LOGICAL =====
        boolean p = true, q = false;
        System.out.println("p && q: " + (p && q));   // false (AND)
        System.out.println("p || q: " + (p || q));   // true  (OR)
        System.out.println("!p    : " + (!p));        // false (NOT)

        // Short-circuit: && if left is false → right is skipped
        int d = 0;
        boolean safe = (d != 0) && (100 / d > 5); // No division by zero!

        // ===== BITWISE =====
        int bw1 = 0b1010, bw2 = 0b1100;
        System.out.println("AND: " + Integer.toBinaryString(bw1 & bw2));  // 1000
        System.out.println("OR : " + Integer.toBinaryString(bw1 | bw2));  // 1110
        System.out.println("XOR: " + Integer.toBinaryString(bw1 ^ bw2));  // 0110
        System.out.println("<<2: " + (bw1 << 2));                         // 40

        // ===== COMPOUND ASSIGNMENT =====
        int s = 100;
        s += 20;  // s = s + 20 → 120
        s -= 10;  // 110
        s *= 2;   // 220
        s /= 4;   // 55
        s %= 10;  // 5

        // ===== TERNARY =====
        int score = 7;
        String result = (score >= 5) ? "Pass" : "Fail";

        // ===== PRECEDENCE =====
        // ++/-- → ! → * / % → + - → < > → == != → && → || → =
        System.out.println("2 + 3 * 4 = " + (2 + 3 * 4));     // 14
        System.out.println("(2+3) * 4 = " + ((2 + 3) * 4));   // 20
    }
}
```

1.2.8. The Math Class — Common Mathematical Functions

MathClassDemo.java

```java
public class MathClassDemo {
    public static void main(String[] args) {
        // Constants
        System.out.println("Math.PI = " + Math.PI);
        System.out.println("Math.E  = " + Math.E);

        // Absolute value, rounding
        System.out.println("abs(-7.5)   = " + Math.abs(-7.5));     // 7.5
        System.out.println("ceil(4.2)   = " + Math.ceil(4.2));     // 5.0
        System.out.println("floor(4.8)  = " + Math.floor(4.8));    // 4.0
        System.out.println("round(4.5)  = " + Math.round(4.5));    // 5

        // Power, square root, logarithm
        System.out.println("pow(2, 10)  = " + Math.pow(2, 10));    // 1024.0
        System.out.println("sqrt(144)   = " + Math.sqrt(144));     // 12.0
        System.out.println("log10(1000) = " + Math.log10(1000));   // 3.0

        // Min, max, random
        System.out.println("max(10, 20) = " + Math.max(10, 20));
        System.out.println("random()    = " + Math.random());      // [0.0, 1.0)

        // Random integer in range [min, max]
        int min = 1, max = 100;
        int randomNum = (int)(Math.random() * (max - min + 1)) + min;

        // Application: distance between two points
        double dist = Math.sqrt(Math.pow(3 - 0, 2) + Math.pow(4 - 0, 2));
        System.out.printf("Distance (0,0) to (3,4) = %.2f%n", dist); // 5.00
    }
}
```

1.2.9. Wrapper Classes

Primitive	Wrapper	Primitive	Wrapper
byte	Byte	float	Float
short	Short	double	Double
int	Integer	char	Character
long	Long	boolean	Boolean

WrapperClasses.java

```java
public class WrapperClasses {
    public static void main(String[] args) {
        // Autoboxing: primitive → object (automatic)
        Integer numObj = 42;            // int → Integer.valueOf(42)
        Double scoreObj = 8.5;

        // Unboxing: object → primitive (automatic)
        int primitiveNum = numObj;      // Integer → int

        // String ↔ Number conversion (VERY COMMONLY USED!)
        int fromString = Integer.parseInt("12345");
        double scoreFromStr = Double.parseDouble("9.75");
        String fromNumber = String.valueOf(42);

        // ⚠️ CAUTION: comparing Integer with == can be misleading
        Integer a = 127, b = 127;
        Integer c = 128, d = 128;
        System.out.println("127 == 127? " + (a == b));     // true (cached: -128..127)
        System.out.println("128 == 128? " + (c == d));     // false! (outside cache)
        System.out.println("128.equals? " + c.equals(d));  // true ← ALWAYS USE THIS!
    }
}
```

> ⚠️ **Important Lesson**
>
> **ALWAYS use** `.equals()` when comparing Wrapper objects! The `==` operator only compares references (memory addresses), not values.

1.2.10. Reading Input with Scanner

ReadingInput.java

```java
import java.util.Scanner;

public class ReadingInput {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter full name: ");
        String fullName = scanner.nextLine();

        System.out.print("Enter age: ");
        int age = scanner.nextInt();

        System.out.print("Enter GPA: ");
        double gpa = scanner.nextDouble();

        // !!! COMMON BUG: nextInt()/nextDouble() leaves '\n' in the buffer
        scanner.nextLine();  // "Consume" the leftover '\n' character

        System.out.print("Enter hometown: ");
        String hometown = scanner.nextLine();  // Now works correctly

        // Formatted output
        System.out.printf("%-10s: %s%n", "Name", fullName);
        System.out.printf("%-10s: %d%n", "Age", age);
        System.out.printf("%-10s: %.2f%n", "GPA", gpa);
        System.out.printf("%-10s: %s%n", "Hometown", hometown);

        // Alternative technique: nextLine() + parse (avoids buffer issues)
        System.out.print("Enter birth year: ");
        int birthYear = Integer.parseInt(scanner.nextLine().trim());

        scanner.close();
    }
}
```

Formatted Output with `printf()`

Specifier	Meaning	Example
`%s`	String	`printf("%s", "Java")`
`%d`	Integer (decimal)	`printf("%d", 42)`
`%f`	Floating-point (float/double)	`printf("%.2f", 3.14)` → 3.14
`%n`	Newline (cross-platform)	
`%-20s`	Left-aligned, min 20 characters	
`%06d`	Zero-padded, min 6 digits	`printf("%06d", 42)` → 000042

1.3. CONTROL FLOW STATEMENTS

1.3.1. Conditional Branching: if / else if / else

IfStatement.java

```java
import java.util.Scanner;

public class IfStatement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter score (0-10 scale): ");
        double score = scanner.nextDouble();

        // --- Input validation ---
        if (score < 0 || score > 10) {
            System.out.println("Invalid score!");
            scanner.close();
            return;
        }

        // --- if-else ---
        if (score >= 5) {
            System.out.println("Result: PASS");
        } else {
            System.out.println("Result: FAIL");
        }

        // --- if-else if-else (grade classification) ---
        String classification;
        double gpa4;

        if (score >= 9.0) {
            classification = "Excellent (A+)"; gpa4 = 4.0;
        } else if (score >= 8.5) {
            classification = "Very Good (A)"; gpa4 = 3.7;
        } else if (score >= 8.0) {
            classification = "Good (B+)"; gpa4 = 3.5;
        } else if (score >= 7.0) {
            classification = "Fairly Good (B)"; gpa4 = 3.0;
        } else if (score >= 5.5) {
            classification = "Average (C)"; gpa4 = 2.0;
        } else if (score >= 4.0) {
            classification = "Below Average (D)"; gpa4 = 1.0;
        } else {
            classification = "Poor (F)"; gpa4 = 0.0;
        }

        System.out.printf("Score: %.1f → %s → GPA 4.0: %.1f%n", score, classification, gpa4);
        scanner.close();
    }
}
```

> ⚠️ **Common Mistake**
>
> Forgetting curly braces `{}` when there are multiple statements in an if block. **Recommendation:** always use `{}` even for single-line bodies.

1.3.2. The switch Statement

SwitchStatement.java

```java
import java.util.Scanner;

public class SwitchStatement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter month (1-12): ");
        int month = scanner.nextInt();

        // ===== TRADITIONAL SWITCH =====
        switch (month) {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                System.out.println("31 days");
                break;                          // break is MANDATORY!
            case 4: case 6: case 9: case 11:
                System.out.println("30 days");
                break;
            case 2:
                System.out.println("28 or 29 days");
                break;
            default:
                System.out.println("Invalid month!");
        }

        // ===== SWITCH EXPRESSION – modern syntax (JDK 14+) =====
        String daysInfo = switch (month) {
            case 1, 3, 5, 7, 8, 10, 12 -> "31 days";
            case 4, 6, 9, 11           -> "30 days";
            case 2                     -> "28 or 29 days";
            default                    -> "Invalid month!";
        };
        System.out.println("[Modern] " + daysInfo);

        // ===== SWITCH WITH BLOCK AND yield =====
        String description = switch (month / 4 + 1) {
            case 1 -> {
                System.out.println("  → Warm spring weather...");
                yield "Spring";
            }
            default -> "Other";
        };

        // ===== SWITCH WITH STRING (JDK 7+) =====
        String day = "Mon";
        String dayType = switch (day.toLowerCase()) {
            case "mon", "tue", "wed", "thu", "fri" -> "Weekday";
            case "sat", "sun"                      -> "Weekend";
            default                                -> "Invalid";
        };

        scanner.close();
    }
}
```

> 💡 **Traditional switch vs Switch Expression**
>
> Switch expressions (arrow `->` syntax) are preferred: no `break` needed, more concise, and safer.
>
> **Valid types:** `byte`, `short`, `int`, `char`, `String`, `enum`. **NOT** supported: `long`, `float`, `double`, `boolean`.

1.3.3. The for Loop

ForLoop.java

```java
public class ForLoop {
    public static void main(String[] args) {
        // ===== BASIC for =====
        System.out.println("=== 7 Times Table ===");
        for (int i = 1; i <= 10; i++) {
            System.out.printf("7 x %2d = %2d%n", i, 7 * i);
        }

        // ===== NESTED for — Number Triangle =====
        int rows = 5;
        for (int i = 1; i <= rows; i++) {
            for (int j = 0; j < rows - i; j++) {
                System.out.print("  ");
            }
            for (int j = 1; j <= 2 * i - 1; j++) {
                System.out.printf("%2d", j);
            }
            System.out.println();
        }

        // ===== TWO CONTROL VARIABLES =====
        for (int i = 0, j = 10; i < j; i++, j--) {
            System.out.println("i=" + i + ", j=" + j);
        }

        // ===== COMPUTATIONS =====
        int sum = 0;
        for (int i = 1; i <= 100; i++) sum += i;
        System.out.println("Sum 1 to 100 = " + sum);  // 5050

        long factorial = 1;
        for (int i = 2; i <= 10; i++) factorial *= i;
        System.out.println("10! = " + factorial);      // 3628800
    }
}
```

1.3.4. while and do-while Loops

WhileLoops.java

```java
import java.util.Scanner;

public class WhileLoops {
    public static void main(String[] args) {
        // ===== while: checks condition BEFORE each iteration =====
        // Euclid's algorithm for GCD
        int a = 48, b = 18;
        int tempA = a, tempB = b;
        while (tempB != 0) {
            int temp = tempB;
            tempB = tempA % tempB;
            tempA = temp;
        }
        System.out.println("GCD(" + a + ", " + b + ") = " + tempA);

        // ===== do-while: executes AT LEAST ONCE =====
        Scanner scanner = new Scanner(System.in);
        int score;
        do {
            System.out.print("Enter score (0-10): ");
            score = scanner.nextInt();
            if (score < 0 || score > 10) {
                System.out.println("  → Invalid! Try again.");
            }
        } while (score < 0 || score > 10);
        System.out.println("Valid score: " + score);

        // ===== MENU-DRIVEN PROGRAM =====
        int choice;
        do {
            System.out.println("\n======= MENU =======");
            System.out.println("1. View information");
            System.out.println("2. Add data");
            System.out.println("3. Edit data");
            System.out.println("0. Exit");
            System.out.print("Choose: ");
            choice = scanner.nextInt();

            switch (choice) {
                case 1 -> System.out.println("→ Displaying...");
                case 2 -> System.out.println("→ Adding...");
                case 3 -> System.out.println("→ Editing...");
                case 0 -> System.out.println("→ Goodbye!");
                default -> System.out.println("→ Invalid option!");
            }
        } while (choice != 0);

        scanner.close();
    }
}
```

> 💡 **When to use while vs do-while?**
>
> **while:** when the loop body may not need to execute at all (e.g., iterating a possibly empty list).
>
> **do-while:** when the body must execute at least once (e.g., menu display, input validation).

1.3.5. break, continue, and labeled break

BreakContinue.java

```java
public class BreakContinue {
    public static void main(String[] args) {
        // ===== break: exits the loop IMMEDIATELY =====
        for (int i = 50; i <= 100; i++) {
            if (i % 7 == 0) {
                System.out.println("First multiple of 7 in [50,100]: " + i);
                break;
            }
        }

        // ===== continue: skips the CURRENT iteration =====
        System.out.print("Odd numbers 1 to 20: ");
        for (int i = 1; i <= 20; i++) {
            if (i % 2 == 0) continue;
            System.out.print(i + " ");
        }
        System.out.println();

        // ===== Labeled break: exits the OUTER loop =====
        int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int target = 5;

        search:
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (matrix[i][j] == target) {
                    System.out.printf("Found %d at [%d][%d]%n", target, i, j);
                    break search;  // Exits BOTH loops
                }
            }
        }
    }
}
```

1.3.6. Comprehensive Example: Prime Number Checker

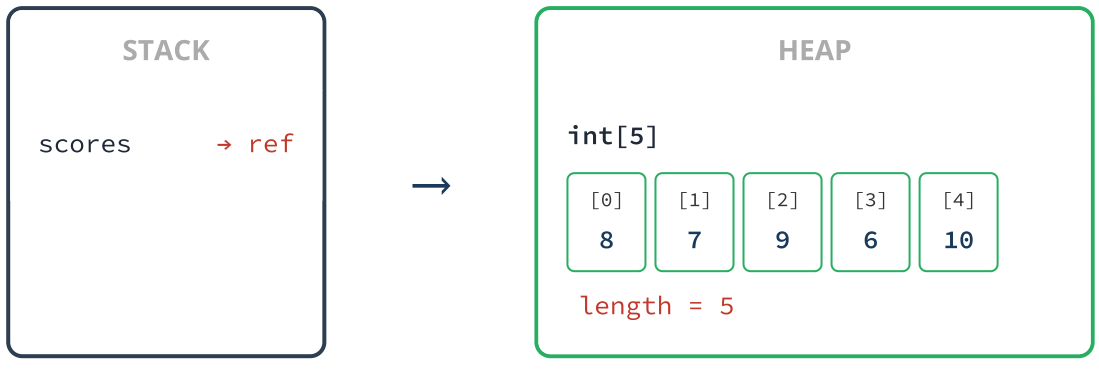
PrimeChecker.java

```java
import java.util.Scanner;

public class PrimeChecker {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a positive integer n: ");
        int n = scanner.nextInt();

        // --- Check if n is a prime number ---
        boolean isPrime = true;

        if (n < 2) {
            isPrime = false;
        } else if (n > 2) {
            if (n % 2 == 0) {
                isPrime = false;
            } else {
                // Only need to check up to √n — a key optimization!
                for (int i = 3; i <= Math.sqrt(n); i += 2) {
                    if (n % i == 0) {
                        isPrime = false;
                        break;
                    }
                }
            }
        }

        System.out.println(n + (isPrime ? " IS" : " is NOT") + " a prime number.");

        // --- List all primes ≤ n ---
        if (n >= 2) {
            System.out.print("Primes ≤ " + n + ": ");
            int count = 0;
            for (int candidate = 2; candidate <= n; candidate++) {
                boolean candidateIsPrime = true;
                for (int i = 2; i <= Math.sqrt(candidate); i++) {
                    if (candidate % i == 0) { candidateIsPrime = false; break; }
                }
                if (candidateIsPrime) { System.out.print(candidate + " "); count++; }
            }
            System.out.println("\n(Total: " + count + " primes)");
        }

        scanner.close();
    }
}
```

1.4. ARRAYS AND STRINGS

1.4.1. One-Dimensional Arrays

An **array** is a data structure that stores a collection of elements of the **same type**, accessed via an **index** starting from **0**. In Java, arrays are **objects** on the **Heap** with a **fixed size** after initialization.



OneDimensionalArray.java

```java
import java.util.Arrays;

public class OneDimensionalArray {
    public static void main(String[] args) {
        // ===== DECLARATION AND INITIALIZATION =====
        int[] scores = new int[5];          // Method 1: size, defaults to 0
        scores[0] = 8; scores[1] = 7; scores[2] = 9;
        scores[3] = 6; scores[4] = 10;

        double[] weights = {1.0, 1.5, 2.0}; // Method 2: literal initialization
        String[] courses = new String[]{"Java", "Python", "C++"};  // Method 3

        // ===== ACCESSING ELEMENTS =====
        System.out.println("Number of courses: " + courses.length);          // 3 (NO parentheses)
        System.out.println("First: " + courses[0]);                          // Java
        System.out.println("Last: " + courses[courses.length - 1]);          // C++
        // courses[3]  → ArrayIndexOutOfBoundsException!

        // ===== TRAVERSING AN ARRAY =====
        // Method 1: traditional for (when you need the index)
        for (int i = 0; i < scores.length; i++) {
            System.out.println("Student " + (i + 1) + ": " + scores[i]);
        }
        // Method 2: for-each (when you only need the value)
        for (String course : courses) {
            System.out.println("  - " + course);
        }

        // ===== THE Arrays UTILITY CLASS =====
        int[] arr = {5, 2, 8, 1, 9, 3};
        System.out.println("Original: " + Arrays.toString(arr));
        Arrays.sort(arr);
        System.out.println("Sorted  : " + Arrays.toString(arr));

        int pos = Arrays.binarySearch(arr, 8);          // Requires sorted array
        int[] copy = Arrays.copyOf(arr, arr.length);
        int[] subArr = Arrays.copyOfRange(arr, 1, 4);  // [1, 4)
        Arrays.fill(new int[3], -1);                    // {-1, -1, -1}
        System.out.println("Equal? " + Arrays.equals(arr, copy));

        // ===== CAUTION: ARRAY ASSIGNMENT COPIES THE REFERENCE =====
        int[] a = {1, 2, 3};
        int[] b = a;           // b and a → point to the SAME array!
        b[0] = 999;
        System.out.println("a[0] = " + a[0]);  // 999! (affected)

        int[] c = Arrays.copyOf(a, a.length);  // True copy
        c[0] = 111;
        System.out.println("a[0] = " + a[0]);  // 999 (not affected)
    }
}
```

1.4.2. Two-Dimensional Arrays

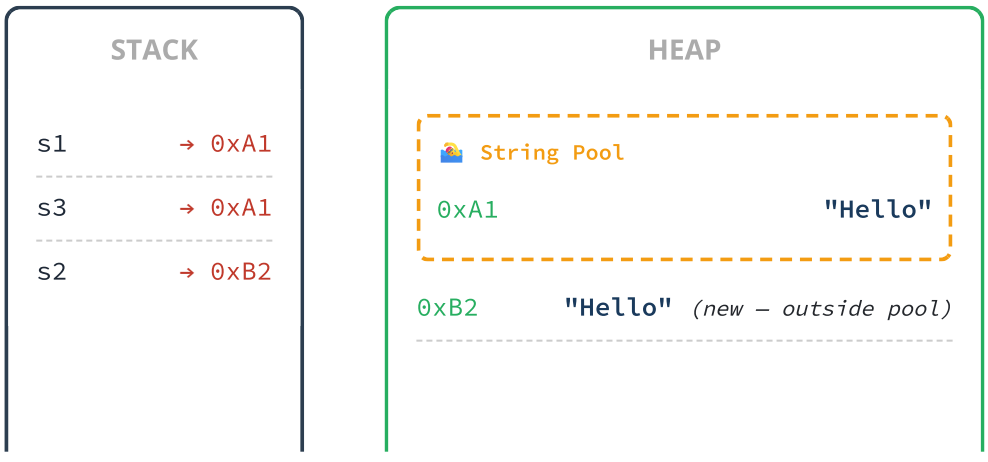
TwoDimensionalArray.java

```java
import java.util.Arrays;

public class TwoDimensionalArray {
    public static void main(String[] args) {
        // ===== INITIALIZATION =====
        int[][] matrix = {
            { 1,  2,  3,  4},
            { 5,  6,  7,  8},
            { 9, 10, 11, 12}
        };
        double[][] gradeTable = new double[3][4];  // 3 rows × 4 columns

        // ===== TRAVERSAL AND PRINTING =====
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.printf("%4d", matrix[i][j]);
            }
            System.out.println();
        }

        // for-each
        int sum = 0;
        for (int[] row : matrix) {
            for (int val : row) sum += val;
        }
        System.out.println("Total sum: " + sum);

        // ===== JAGGED ARRAY — rows with different lengths =====
        int[][] pascalTriangle = new int[6][];
        for (int i = 0; i < pascalTriangle.length; i++) {
            pascalTriangle[i] = new int[i + 1];
            pascalTriangle[i][0] = 1;
            pascalTriangle[i][i] = 1;
            for (int j = 1; j < i; j++) {
                pascalTriangle[i][j] = pascalTriangle[i - 1][j - 1]
                                     + pascalTriangle[i - 1][j];
            }
        }

        System.out.println("\nPascal's Triangle:");
        for (int[] row : pascalTriangle) {
            for (int val : row) System.out.printf("%4d", val);
            System.out.println();
        }

        System.out.println("deepToString: " + Arrays.deepToString(matrix));
    }
}
```

1.4.3. The String Type

`String` is a reference type but is given special treatment in Java. Its most important characteristic: **String is immutable** — once created, its content cannot be changed. Every method that appears to "modify" a String actually creates a new one.

The String Pool



```
s1 == s3 → true (same ref)
s1 == s2 → false (different ref)
s1.equals(s2) → true (same content)
```

StringDemo.java

```java
public class StringDemo {
    public static void main(String[] args) {
        // ===== CREATION =====
        String s1 = "Hello";                    // Literal → String Pool
        String s2 = new String("Hello");         // new → separate object
        String s3 = "Hel" + "lo";               // Concatenated literals → pool

        // ===== COMPARISON — MUST use .equals() =====
        System.out.println("s1 == s3     : " + (s1 == s3));           // true
        System.out.println("s1 == s2     : " + (s1 == s2));           // false!
        System.out.println("s1.equals(s2): " + s1.equals(s2));       // true
        System.out.println("equalsIgnoreCase: " + "java".equalsIgnoreCase("JAVA"));

        // ★★★ GOLDEN RULE: ALWAYS use .equals() to compare String content ★★★

        // ===== COMMONLY USED METHODS =====
        String text = "  Java Programming at Nha Trang University  ";

        // Information
        System.out.println("length()  : " + text.length());
        System.out.println("charAt(5) : " + text.charAt(5));
        System.out.println("isEmpty() : " + text.isEmpty());
        System.out.println("isBlank() : " + "   ".isBlank());     // JDK 11

        // Searching
        System.out.println("indexOf   : " + text.indexOf("Java"));
        System.out.println("contains  : " + text.contains("Nha Trang"));
        System.out.println("startsWith: " + text.startsWith("  Java"));

        // Transformation (creates a NEW String)
        System.out.println("trim()     : '" + text.trim() + "'");
        System.out.println("strip()    : '" + text.strip() + "'");  // JDK 11
        System.out.println("toUpperCase: " + text.trim().toUpperCase());
        System.out.println("replace    : " + text.trim().replace("Java", "Python"));
        System.out.println("substring  : " + text.trim().substring(5, 16));

        // ===== SPLITTING AND JOINING =====
        String csv = "Java,Python,C++,JavaScript";
        String[] languages = csv.split(",");
        for (String lang : languages) System.out.println("  - " + lang);
        System.out.println("Joined: " + String.join(" | ", languages));

        // ===== TYPE CONVERSION =====
        String fromNum = String.valueOf(2024);
        int numFromStr = Integer.parseInt("42");
        double scoreFromStr = Double.parseDouble("9.5");

        // ===== TEXT BLOCKS (JDK 15+) =====
        String json = """
                {
                    "name": "John Doe",
                    "age": 20,
                    "major": "Computer Science"
                }
                """;
        System.out.println(json);

        // ===== FORMATTED STRING =====
        String info = "Student: %s, GPA: %.2f".formatted("Alice", 3.75);

        // ===== IMMUTABILITY DEMONSTRATED =====
        String original = "Hello";
        String modified = original.concat(" World");
        System.out.println("original: " + original);   // "Hello" — UNCHANGED
        System.out.println("modified: " + modified);    // "Hello World" — new String
    }
}
```

1.4.4. StringBuilder — Mutable Strings

Since `String` is **immutable**, concatenating strings inside a loop creates many temporary objects → slow. `StringBuilder` allows modifying the content in place within the same memory region.

StringBuilderDemo.java

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        // String concatenation in a loop: O(n²)  ← SLOW
        // StringBuilder:                  O(n)   ← FAST

        StringBuilder sb = new StringBuilder();
        sb.append("Student List:\n");

        String[] names = {"Alice Johnson", "Bob Smith", "Charlie Brown"};
        for (int i = 0; i < names.length; i++) {
            sb.append("  ").append(i + 1).append(". ").append(names[i]).append("\n");
        }

        String result = sb.toString();
        System.out.println(result);

        // Key methods
        StringBuilder sb2 = new StringBuilder("Hello Java");
        sb2.insert(5, " Beautiful");      // "Hello Beautiful Java"
        sb2.delete(5, 16);                // "Hello Java"
        sb2.replace(6, 10, "World");      // "Hello World"
        sb2.reverse();                    // "dlroW olleH"

        System.out.println("length  : " + sb2.length());
        System.out.println("capacity: " + sb2.capacity());
    }
}
```

> 💡 **When to use which?**
>
> **String:** strings that rarely change, method parameters, keys in a Map.
> **StringBuilder:** building strings in loops, concatenating many fragments.
> **StringBuffer:** same as StringBuilder but **thread-safe** (slower) → use in multithreaded environments.

1.4.5. Comprehensive Example: Array and String Processing

ComprehensiveExample.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class ComprehensiveExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of students: ");
        int n = Integer.parseInt(sc.nextLine().trim());

        String[] names = new String[n];
        double[] scores = new double[n];

        for (int i = 0; i < n; i++) {
            System.out.printf("\n--- Student %d ---%n", i + 1);
            System.out.print("  Name : ");
            names[i] = sc.nextLine().trim();
            System.out.print("  Score: ");
            scores[i] = Double.parseDouble(sc.nextLine().trim());
        }

        // Find max, min, average
        double max = scores[0], min = scores[0], total = 0;
        int maxIdx = 0, minIdx = 0;

        for (int i = 0; i < n; i++) {
            total += scores[i];
            if (scores[i] > max) { max = scores[i]; maxIdx = i; }
            if (scores[i] < min) { min = scores[i]; minIdx = i; }
        }

        // Sort by score descending
        Integer[] indices = new Integer[n];
        for (int i = 0; i < n; i++) indices[i] = i;
        Arrays.sort(indices, (a, b) -> Double.compare(scores[b], scores[a]));

        // Display grade report
        System.out.println("\n╔═══════╦══════════════════════╦═══════╦══════════╗");
        System.out.println("║ No.   ║     Full Name        ║ Score ║  Grade   ║");
        System.out.println("╠═══════╬══════════════════════╬═══════╬══════════╣");
        for (int idx : indices) {
            String grade = scores[idx] >= 8 ? "Good" :
                           scores[idx] >= 6.5 ? "Fair" :
                           scores[idx] >= 5 ? "Avg" : "Poor";
            System.out.printf("║ %3d ║ %-16s ║ %5.1f ║ %-8s║%n",
                              idx + 1, names[idx], scores[idx], grade);
        }
        System.out.println("╚═══════╩══════════════════════╩═══════╩══════════╝");
        System.out.printf("Highest : %.1f (%s)%n", max, names[maxIdx]);
        System.out.printf("Lowest  : %.1f (%s)%n", min, names[minIdx]);
        System.out.printf("Average : %.2f%n", total / n);

        sc.close();
    }
}
```

PRACTICE EXERCISES — CHAPTER 1

🟢 Basic Level

Exercise 1.1 — Simple Calculator: Read two real numbers and an operator (+, -, *, /, %), display the result. Handle division by zero and invalid operators. Use switch expressions.

Exercise 1.2 — Temperature Converter: Read a temperature and its unit (C/F/K), convert to the other two units and display.

Exercise 1.3 — Leap Year Checker: Read a year and determine whether it is a leap year (divisible by 4 but not 100, except when divisible by 400).

🟡 Intermediate Level

Exercise 1.4 — Number Analysis: Read a positive integer, list its divisors, check if it is a perfect number (e.g., 6 = 1+2+3), and find its prime factorization.

Exercise 1.5 — Array Statistics: Read an array of n real numbers, compute max, min, mean, variance, and standard deviation. Count elements greater than the mean.

Exercise 1.6 — Advanced String Processing: Read a sentence, count words, count vowels/consonants, reverse word order, convert to Title Case, and remove extra whitespace.

🔴 Advanced Level

Exercise 1.7 — Sieve of Eratosthenes: Find all primes ≤ N using the Sieve of Eratosthenes algorithm (boolean array).

Exercise 1.8 — Spiral Matrix: Generate an n×n matrix filled with numbers 1 to n^2 in a spiral pattern.

Exercise 1.9 — Number Guessing Game: The computer generates a random number 1–100; the user guesses with "higher"/"lower" hints. Count the number of attempts.

Sample Solution — Exercise 1.1

SimpleCalculator.java

```java
import java.util.Scanner;

public class SimpleCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        double a = Double.parseDouble(sc.nextLine().trim());

        System.out.print("Enter operator (+, -, *, /, %): ");
        char operator = sc.nextLine().trim().charAt(0);

        System.out.print("Enter second number: ");
        double b = Double.parseDouble(sc.nextLine().trim());

        String result = switch (operator) {
            case '+' -> String.format("%.4g + %.4g = %.4g", a, b, a + b);
            case '-' -> String.format("%.4g - %.4g = %.4g", a, b, a - b);
            case '*' -> String.format("%.4g × %.4g = %.4g", a, b, a * b);
            case '/' -> {
                if (b == 0) yield "Error: Cannot divide by zero!";
                yield String.format("%.4g ÷ %.4g = %.4g", a, b, a / b);
            }
            case '%' -> {
                if (b == 0) yield "Error: Cannot divide by zero!";
                yield String.format("%.4g %% %.4g = %.4g", a, b, a % b);
            }
            default -> "Operator '" + operator + "' is invalid!";
        };

        System.out.println("Result: " + result);
        sc.close();
    }
}
```

REVIEW QUESTIONS (Self-Assessment)

Q1: Explain the differences between JDK, JRE, and JVM. When do you need the JDK, and when is the JRE sufficient?

Q2: Is Java a compiled language or an interpreted language? Explain your reasoning.

Q3: What is the output of the following code? Explain step by step.

```
int a = 5;
int b = a++ + ++a;
System.out.println("a = " + a + ", b = " + b);
```

Q4: Why does `0.1 + 0.2 != 0.3` in Java? How can you work around this?

Q5: What is the difference between `==` and `.equals()` when comparing strings?

Q6: What is the output?

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1 == s3);
System.out.println(s1.equals(s3));
```

Q7: What are the differences between `String` and `StringBuilder`? When should you use `StringBuilder`?

Q8: Why does the following code cause a compilation error?

```
byte b1 = 10, b2 = 20;
byte b3 = b1 + b2;
```

Q9: What is the difference between `while` and `do-while`? Give a practical example for each.

Q10: What does this code print? Explain why.

```
int[] arr = {1, 2, 3};
int[] ref = arr;
ref[0] = 99;
System.out.println(arr[0]);
```

📝 CHAPTER SUMMARY

This chapter establishes a solid foundation in the Java programming language, covering: Java's history and key features along with its platform editions (SE, EE, ME); the JDK/JRE/JVM architecture and the two-phase compilation–execution process; how to set up the development environment and use JShell.

Core content: the system of 8 primitive data types with wrapper classes; the Stack/Heap memory model; a complete operator system; type casting rules and type promotion in expressions; control flow statements (if/else, traditional and expression switch, for, while, do-while, break/continue); the Math class; input/output with Scanner and printf.

The final section covers: one-dimensional arrays, two-dimensional arrays, String (immutability, String Pool), and StringBuilder. This knowledge is an essential prerequisite for **Chapter 2 — Object-Oriented Programming**.

📚 References

1. Y. Daniel Liang, *Introduction to Java Programming and Data Structures*, Comprehensive Version, 12th Ed., Pearson, 2020. — Chapters 1–7.

2. Doan Van Ban, Doan Van Trung, *Giáo trình lập trình Java* (Java Programming Textbook), Vietnam Education Publisher, 2014.

3. Cay S. Horstmann, *Core Java, Volume I — Fundamentals*, 12th Ed., Oracle Press, 2022. — Chapters 1–3.

4. Oracle, *The Java Tutorials*: docs.oracle.com/javase/tutorial

5. Oracle, *Java SE 23 API Documentation*: docs.oracle.com/en/java/javase/23/docs/api

---

English edition for academic English enhancement purposes